Editorial: Problem B - Sayaka's Blend

Setter: Hudson

Problem Overview

We are given N items, each with value F_i and weight C_i. We need to choose a non-empty subset S such that $\frac{\sum_{i \in S} C_i}{|S|} = K$, maximizing $\sum_{i \in S} F_i$.

Difficulty Analysis

Rating: **2100**. Constraints: $N \leq 40$. $O(2^N)$ is too slow. This suggests a **Meet-in-the-Middle** approach.

Solution Approach

First, transform the condition.

$$\frac{\sum C_i}{|S|} = K \iff \sum C_i = K \cdot |S| \iff \sum (C_i - K) = 0$$

Let $W_i = C_i - K$. The problem becomes: Find a subset with sum of weights W_i equal to 0, maximizing sum of F_i.

Algorithm: 1. Split the N items into two halves: Left ($N/2$) and Right ($N - N/2$). 2. Generate all $2^{N/2}$ subset sums for the Left half. Store them in a map: 'Map[WeightSum] = max(FlavorSum)'. (Note: Track whether the subset is empty or not, or just ensure we check non-empty combinations later). 3. Generate all subset sums for the Right half. 4. For each Right subset with weight sum W_{right} and flavor F_{right}: We need a Left subset with weight $W_{left} = -W_{right}$. If such a key exists in the map, potential answer is $F_{right} + Map[-W_{right}]$. 5. Be careful to ensure the total subset size > 0. Since we want to maximize flavor (and flavor is positive), the only risk is picking empty Left + empty Right, which sums to 0 but is invalid. We can just ignore the (0, 0) case or handle it explicitly.

Complexity: $O(2^{N/2} \cdot \log(2^{N/2})) = O(2^{N/2} \cdot N)$. With $N = 40$, $2^{20} \approx 10^6$, which fits easily in 2.0s.

Implementation (C++)

```cpp
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>

using namespace std;

typedef long long ll;
const ll INF = 1e18;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    ll k;
    if (!(cin >> n >> k)) return 0;

    vector<pair<ll, ll>> items(n);
    for (int i = 0; i < n; ++i) {
        cin >> items[i].first >> items[i].second; // F, C
        items[i].second -= k; // Transform C -> C - K
    }

    int n1 = n / 2;
    int n2 = n - n1;

    // Map stores: Normalized Caffeine Sum -> Max Flavor Sum
    map<ll, ll> left_map;

    // Generate Left subsets
    for (int i = 0; i < (1 << n1); ++i) {
        ll current_w = 0;
        ll current_f = 0;
        for (int j = 0; j < n1; ++j) {
            if ((i >> j) & 1) {
                current_f += items[j].first;
                current_w += items[j].second;
            }
        }
        // Store max flavor for this weight sum
        if (left_map.find(current_w) == left_map.end()) {
            left_map[current_w] = current_f;
        } else {
            left_map[current_w] = max(left_map[current_w], current_f);
        }
    }

    ll ans = 0;
    bool found = false;

    // Generate Right subsets
    for (int i = 0; i < (1 << n2); ++i) {
        ll current_w = 0;
        ll current_f = 0;
        bool right_non_empty = false;
```

```
57
58          for (int j = 0; j < n2; ++j) {
59              if ((i >> j) & 1) {
60                  current_f += items[n1 + j].first;
61                  current_w += items[n1 + j].second;
62                  right_non_empty = true;
63              }
64          }
65
66          // We need left_w + current_w = 0  => left_w = -current_w
67          if (left_map.count(-current_w)) {
68              ll total_f = current_f + left_map[-current_w];
69
70              // Check if valid (non-empty)
71              // It is valid if Right is non-empty OR Left is non-empty.
72              // The only invalid case is Right empty AND Left empty (Map[0]=0).
73              // But Map[0] stores 0 (empty set sum).
74              // So if total_f == 0 and we haven't found anything, we shouldn't update.
75              // Better check:
76              if (right_non_empty || (-current_w != 0) || (left_map[-current_w] != 0)) {
77                   // Even simpler: If total subset isn't empty.
78                   // Just check if we are not in the (0,0) empty state.
79                   // Actually, if input F_i are all positive, then total_f > 0 implies
                         non-empty.
80                   // But F_i >= 1.
81                   if (total_f > 0) {
82                       ans = max(ans, total_f);
83                       found = true;
84                   }
85              }
86          }
87      }
88
89      cout << ans << endl;
90
91      return 0;
92  }
```